8

AUTOSTEREOGRAMS

Stare at **Figure 8-1** for a minute. Do you see anything other than random dots? **Figure 8-1** is an *autostereogram*, a two-dimensional image that creates the illusion of three dimensions. Autostereograms usually consist of repeating patterns that resolve into three dimensions on closer inspection. If you can't see any sort of image, don't worry; it took me a while and a bit of experimentation before I could. (If you aren't having any luck with the version printed in this book, try the color version in the *images* folder of the book's GitHub repository. The footnote to the caption reveals what you should see.)

In this project, you'll use Python to create autostereograms. Here are some of the concepts covered in this project:

- Linear spacing and depth perception
- Depth maps
- Creating and editing images using `Pillow`
- Drawing into images using `Pillow`

Figure 8-1: *A puzzling image that might gnaw at you*[1]

The autostereograms you'll generate in this project are designed for "wall-eyed" viewing. The best way to see them is to focus your eyes on a point behind the image (such as a wall). Almost magically, once you perceive something in the patterns, your eyes should automatically bring it into focus, and when the three-dimensional image "locks in," you'll have a hard time shaking it off. (If you're still having trouble viewing the image, see Gene Levin's article "How to View Stereograms and Viewing Practice"[2] for help.)

How It Works

An autostereogram starts as an image with a repeating tiled pattern. The hidden 3D image is embedded into it by changing the linear spacing between the repeating patterns, thereby creating the illusion of depth. When you look at repeating patterns in an autostereogram, your brain can interpret the spacing as depth information, especially if there are multiple patterns with different spacing.

Perceiving Depth in an Autostereogram

When your eyes converge at an imaginary point behind the image, your brain matches the points seen by your left eye with a different

group seen by your right eye, and you see these points lying on a plane behind the image. The perceived distance to this plane depends on the amount of spacing in the pattern. For example, **Figure 8-2** shows three rows of *A*s. The *A*s are equidistant within each row, but their horizontal spacing increases from top to bottom.

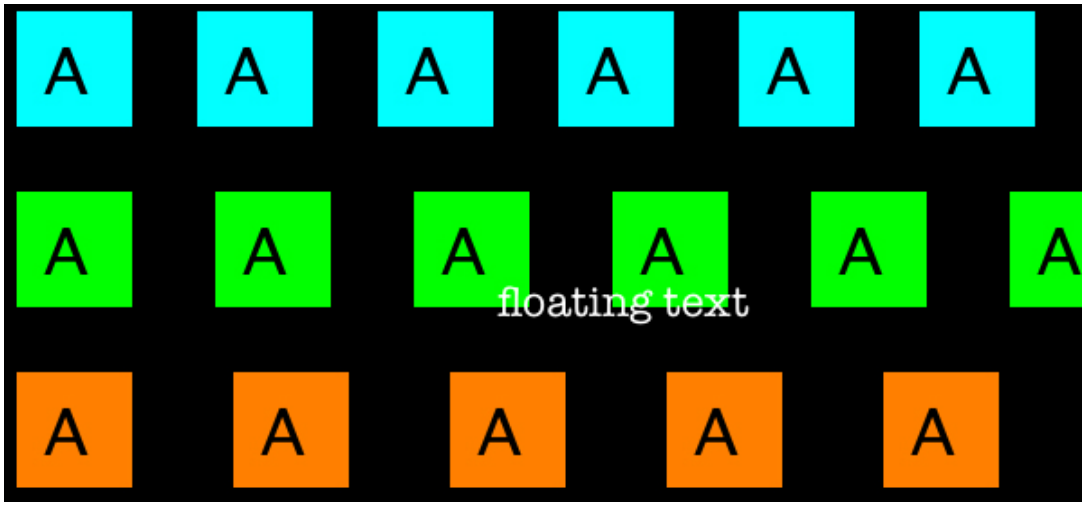


Figure 8-2: *Linear spacing and depth perception*

When this image is viewed "wall-eyed," the top row in **Figure 8-2** should appear to be behind the paper, the middle row should look like it's a little behind the first row, and the bottom row should appear farthest from your eye. The text that says *floating text* should appear to "float" on top of these rows.

Why does your brain interpret the spacing between these patterns as depth? Normally, when you look at a distant object, your eyes work together to focus and converge at the same point, with both eyes rotating inward to point directly at the object. But when viewing a "wall-eyed" autostereogram, focus and convergence happen at different locations. Your eyes focus on the autostereogram, but your brain sees the repeated patterns as coming from the same virtual (imaginary) object, and your eyes converge on a point behind the image, as shown in **Figure 8-3**. This combination of decoupled focus and convergence allows you to see depth in an autostereogram.

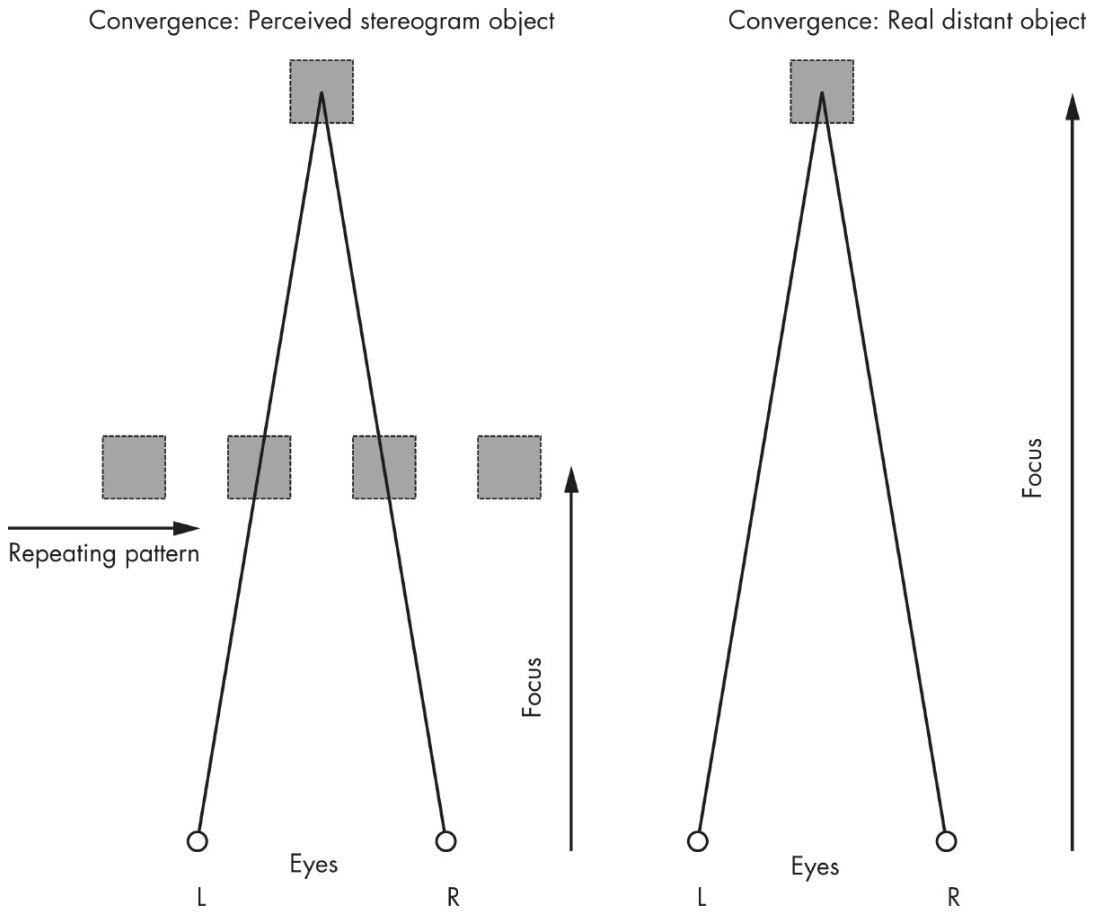


Figure 8-3: *Seeing depth in autostereograms*

The perceived depth of the autostereogram depends on the horizontal spacing of pixels. Because the first row in **Figure 8-2** has the closest spacing, it appears in front of the other rows. However, if the spacing of the points were varied in the image, your brain would perceive each point at a different depth, and you could see a virtual three-dimensional image appear.

Working with Depth Maps

The hidden image in an autostereogram comes from a *depth map*, an image in which the value of each pixel represents a depth value, which is the distance from the eye to the part of the object represented by that pixel. A depth map is often shown as a grayscale image, with light areas for nearby points and darker areas for points farther away, as shown in **Figure 8-4**.

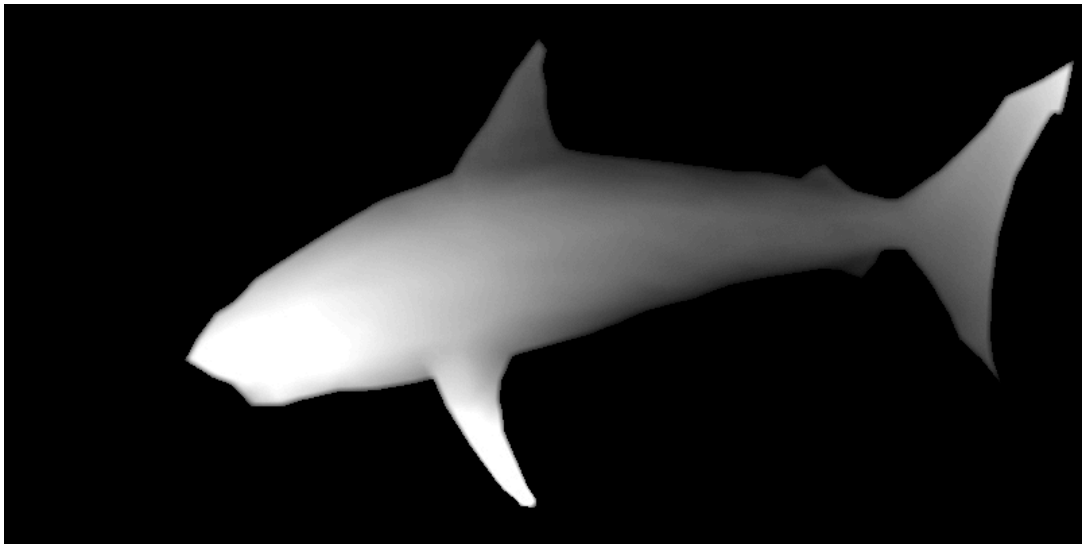


Figure 8-4: *A depth map*

Notice that the nose of the shark, the lightest part of the image, seems closest to you. The darker area toward the tail seems farthest away. (By the way, the image in **Figure 8-4** is the same depth map used to create the first autostereogram shown in **Figure 8-1**.)

Because the depth map represents the depth or distance from the center of each pixel to the eye, you can use it to get the depth value associated with a pixel location in the image. You know that horizontal shifts are perceived as depth in images. So if you shift a pixel in a (patterned) image proportionally to the corresponding pixel's depth value, you would create a depth perception for that pixel consistent with the depth map. If you do this for all pixels, you'll end up encoding the entire depth map into the image, creating an autostereogram.

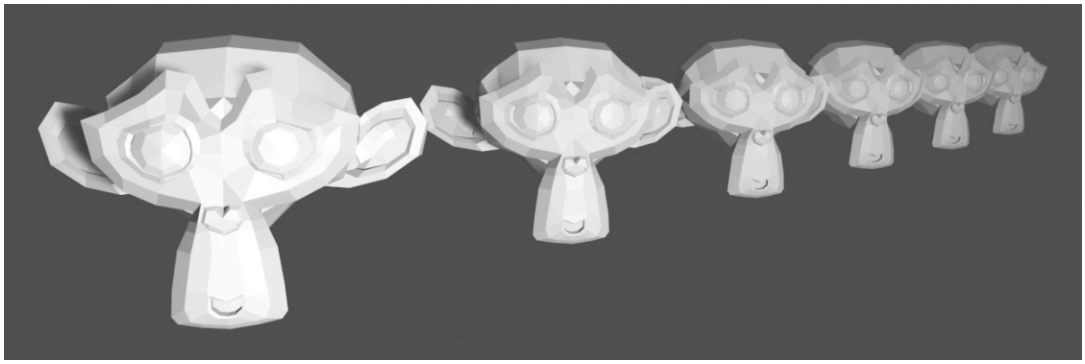
Depth maps store depth values for each pixel, and the resolution of the value depends on the number of bits used to represent it. Because you'll be using common 8-bit images in this chapter, depth values will be in the range [0, 255].

For the purposes of this project, I've posted several example depth maps to the book's GitHub repository. You can download the maps and use them as input for generating autostereograms. However, you may

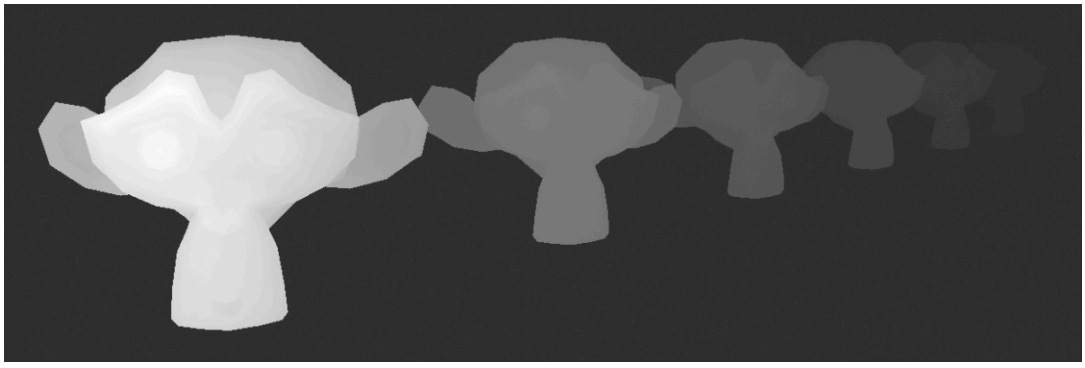
also want to try making your own depth maps to create some fancier images. There are two approaches you can take: using synthetic images created with 3D modeling software or using photographs taken with a smartphone camera.

Creating Depth Maps from 3D Models

If you create a 3D model of something using a 3D computer graphics program like Blender, you can also use the program to generate a depth map of the model. **Figure 8-5** shows an example.



(a)



(b)

Figure 8-5: *A 3D model (a) and its associated depth map (b)*

Figure 8-5(a) shows a 3D model rendered using Blender, and **Figure 8-5(b)** shows the depth map created from this model. Search YouTube for "Blender depth map in 5 minutes!" for a tutorial[3] from

Jonty Schmidt on how to do this. The key is to color the image based on the Z-distance from the camera.

Creating Depth Maps from Smartphone Photographs

These days, many smartphone cameras have a *portrait mode*, which captures depth information along with the photograph to selectively blur out the background. If you could get hold of this depth data, you'd have a depth map of the photograph that you can use to create an autostereogram! **Figure 8-6** shows an example.

(a)

(b)

Figure 8-6: *A portrait mode photo (a) and depth map (b) from an iPhone 11 camera*

Figure 8-6(a) shows a photograph taken in portrait mode with an iPhone 11, and **Figure 8-6(b)** shows the corresponding depth map. The depth map was created with the open source software ExifTool using this command:

```
exiftool -b -MPImage2 photo.jpg > depth.jpg
```

This command extracts the depth information from the metadata in the file *photo.jpg* and saves it to the file *depth.jpg*. Download ExifTool from ***https://exiftool.org*** to try the process for yourself. The command works for photos from an iPhone, but you can use similar techniques to extract the depth data from images taken with other types of phones. There are also various apps available on the Android and iOS app stores that can help you with this. Here's one online depth map extractor that works with portrait mode images from a variety of phone models: ***http://www.hasaranga.com/dmap***.

Shifting Pixels

We've looked at how our brains perceive the spacing between repeating elements in an image as depth information, and we've seen how depth information is conveyed through a depth map. Now let's look at how to shift the pixels in a tiled image in proportion to the values in a depth map. This is the key step in creating an autostereogram.

A tiled image is created by repeating a smaller image (the tile) in the x- and y-directions, although for depth perception, we're concerned only with the x-direction. If the tile that makes up the image is w pixels wide, you know the color values of the image's pixels will repeat every w pixels in the x-direction for any given row. Put another way, the color of the pixel in some row at point i along the x-axis can be expressed as:

$C_i = C_{i-w}$ for $i \geq w$

Let's consider an example. Given a tile width of 100 pixels, for a pixel with an x-axis position of 140, the equation tells you that $C_{140} = C_{140-100} = C_{40}$. This means the color value of the pixel at x-position 140 is the same as that of the pixel at x-position 40, due to the repetition of the image. (For values of i less than w in the previous formula, the color is just C_i, since the tile hasn't repeated yet.)

The goal is to shift pixels in the tiled image according to the values in the depth map. Let δ_i be the value at x-position i in the depth map. The shifted color value of the corresponding pixel in the tiled image is given by:

$$C_i = C_{i-w+\delta_i}$$

Returning to the example where the tile width is 100 pixels, given a pixel at x-position 140 and a corresponding depth map value of 10, the formula says that $C_{140} = C_{140-100+10} = C_{50}$. Because of the depth map, the color of the pixel at position 140 should be changed to match the color of the pixel at position 50. Since C_{50} is the same as C_{150}, this is effectively taking the pixel at x-position 150 and shifting it 10 pixels to the left. As a result, the repetition between positions 50 and 150 has become 10 pixels narrower, and your brain will perceive this change as depth information.

To create the complete autostereogram, you'll repeat this shifting process across the width of the image and over all the rows. You'll see how the shifting is actually implemented when you review the code.

Requirements

In this project, you'll use `Pillow` to read in images, access their underlying data, and create and modify images.

The Code

The code for this project will follow these steps to create an autostereogram:

1. Read in a depth map.
2. Read in a tile image or create a "random dot" tile. This will serve as the basis for the autostereogram's repeating pattern.
3. Create a new image by repeating the tile. The dimensions of this image should match those of the depth map.
4. For each pixel in the new image, shift the pixel by an amount proportional to the depth value associated with the corresponding pixel in the depth map.
5. Write the resulting autostereogram to a file.

To see the complete project, skip ahead to **"The Complete Code"** on **page 147**. You can also download the full code listing for this chapter from ***https://github.com/mkvenkit/pp2e/tree/main/autos***.

Creating a Tile from Random Circles

The user will have the option to provide a tile image at the start of the program (I've uploaded an image based on an M.C. Escher drawing to GitHub for this purpose). If one isn't provided, create a tile with random circles using the `createRandomTile()` function.

```
def createRandomTile(dims):

    # create image

❶ img = Image.new('RGB', dims)

❷ draw = ImageDraw.Draw(img)

    # set the radius of a random circle to 1% of
```

```
    # width or height, whichever is smaller
❸   r = int(min(*dims)/100)
    # number of circles
❹   n = 1000
    # draw random circles
    for i in range(n):
        # -r makes sure that the circles stay inside and aren't cut off
        # at the edges of the image so that they'll look better when tiled
❺       x, y = random.randint(r, dims[0]-r), random.randint(r, dims[1]-r)
❻       fill = (random.randint(0, 255), random.randint(0, 255),
                random.randint(0, 255))
❼       draw.ellipse((x-r, y-r, x+r, y+r), fill)
    return img
```

First you create a new Python Imaging Library (PIL) `Image` object with the dimensions given by `dims` ❶. Then you use `ImageDraw.Draw()` ❷ to draw circles inside the image with an arbitrarily chosen radius (`r`) of 1/100th of either the width or the height of the image, whichever is smaller ❸. (The Python `*` operator unpacks the width and height values in the `dims` tuple so that they can be passed into the `min()` method.)

You set the number of circles to draw to `1000` ❹ and then calculate the x- and y-coordinates of the center of each circle by calling `random.randint()` to get random integers in the range [`r`, `width -`

`r`] and [`r`, `height -r`] ❺. Offsetting the range by `r` ensures the generated circles will fall entirely inside the boundaries of the tile. Without it, you could end up drawing a circle right at the edge of the image, which means it would be partly cut off. If you tiled such an image to create the autostereogram, the result wouldn't look good because the circles at the edge between two tiles would have no spacing between them.

Next, you select a fill color for each circle by randomly choosing RGB values from the range [`0`, `255`] ❻. Finally, you use the `ellipse()` method in `draw` to draw each of your circles ❼. The first argument to this method is a tuple defining the bounding box of the circle, which is given by the top-left and bottom-right corners as (`x-r`, `y-r`) and (`x+r`, `y+r`), respectively, where (`x`, `y`) is the center of the circle and `r` is its radius. The other argument is the randomly chosen fill color.

You can test this method in the Python interpreter as follows:

```
>>> import autos

>>> img = autos.createRandomTile((256, 256))

>>> img.save('out.png')

>>> exit()
```

Figure 8-7 shows the output from the test.

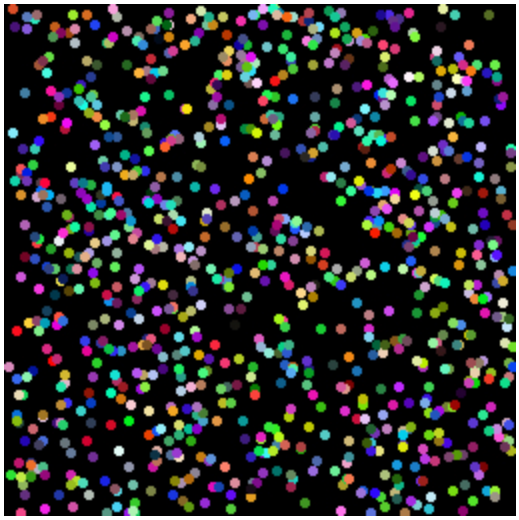


Figure 8-7: *A sample run of* `createRandomTile()`

As you can see in **Figure 8-7**, you've created an image with random dots that you can use as the autostereogram's tiled pattern.

Repeating a Given Tile

Now that you have a tile to work with, you can create an image by repeating that tile. This will form the basis of your autostereogram. Define a `createTiledImage()` function to do the work.

```
def createTiledImage(tile, dims):
    # create the new image
❶   img = Image.new('RGB', dims)
    W, H = dims
    w, h = tile.size
    # calculate the number of tiles needed
❷   cols = int(W/w) + 1
```

```
❸ rows = int(H/h) + 1

  # paste the tiles into the image

  for i in range(rows):

      for j in range(cols):

          ❹ img.paste(tile, (j*w, i*h))

  # output the image

  return img
```

The function takes in an image that will serve as the tiled pattern (`tile`), and the desired dimensions of the output image (`dims`). The dimensions are given as a tuple in the form (`width`, `height`). You create a new `Image` object using the supplied dimensions ❶. Next, you store the width and height of both the individual tile and the overall image. Dividing the overall image dimensions by those of the tile gives you the number of columns ❷ and rows ❸ of tiles you need to have in the image. You add 1 to each calculation to make sure that the last column of tiles on the right and the last row of tiles on the bottom aren't missed when the output image dimension isn't an exact integer multiple of the tile dimension. Without this precaution, the right and bottom of the image might be cut off. Finally, you loop through the rows and columns and fill them with tiles ❹. You determine the location of the top-left corner of the tile by multiplying `(j*w, i*h)` so it aligns with the rows and columns, just as you did in the photomosaic project. Once complete, the function returns an `Image` object of the specified dimensions, tiled with the input image `tile`.

Creating Autostereograms

Now let's create some autostereograms. The `createAutostereogram()` function does most of the work. Here it is:

```
def createAutostereogram(dmap, tile):
    # convert the depth map to a single channel if needed
❶   if dmap.mode != 'L':
        dmap = dmap.convert('L')
    # if no image is specified for a tile, create a random circles tile
❷   if not tile:
        tile = createRandomTile((100, 100))
    # create an image by tiling
❸   img = createTiledImage(tile, dmap.size)
    # create a shifted image using depth map values
❹   sImg = img.copy()
    # get access to image pixels by loading the Image object first
❺   pixD = dmap.load()
    pixS = sImg.load()
    # shift pixels horizontally based on depth map
❻   cols, rows = sImg.size
    for j in range(rows):
        for i in range(cols):
            ❼ xshift = pixD[i, j]/10
```

```
        ❽ xpos = i - tile.size[0] + xshift

        ❾ if xpos > 0 and xpos < cols:

            ❿ pixS[i, j] = pixS[xpos, j]

    # display the shifted image

    return sImg
```

First you convert the provided depth map (`dmap`) into a single-channel grayscale image if needed ❶. If the user doesn't supply an image for the tile, you then create a tile of random circles using the `createRandomTile()` function you defined earlier ❷. Next, you use your `createTiledImage()` function to create a tiled image that matches the size of the supplied depth map image ❸. You then make a copy of this tiled image ❹. This copy will become the final autostereogram.

The function continues by using the `Image.load()` method on the depth map and the output image ❺. This method loads image data into memory, allowing you to access an image's pixels as a two-dimensional array in the form `[i, j]`. You store the image dimensions as a number of rows and columns ❻, treating the image as a grid of individual pixels.

The core of the autostereogram creation algorithm lies in the way you shift the pixels in the tiled image according to the information gathered from the depth map. To do this, you iterate through the tiled image and process each pixel. First you look up the value of the corresponding pixel from the depth map and divide this value by 10 to determine a shift value for the tiled image ❼. You divide by 10 because you're using an 8-bit depth map here, which means the depth varies from 0 to 255. If you divide these values by 10, you get depth values in the approximate range of 0 to 25. Since the depth map input image dimensions are usually in the hundreds of pixels, these shift values

work fine. (Play around by changing the value you divide by to see how it affects the final image.)

Next, you use the formula discussed in **"Shifting Pixels"** on **page 140** to calculate the x-axis coordinate to look for the pixel's new color value ❽. Pixels with a depth map value of 0 (black) won't be shifted and will be perceived as the background. After checking to make sure you're not trying to access a pixel that's not in the image (which can happen at the image's edges because of the shift) ❾, you replace each pixel with its shifted value ❿.

Providing Command Line Options

The `main()` function of the program provides some command line options to customize the autostereogram.

```
def main():
    # create a parser
    parser =
argparse.ArgumentParser(description="Autostereograms...")
    # add expected arguments
❶ parser.add_argument('--depth', dest='dmFile', required=True)
    parser.add_argument('--tile', dest='tileFile', required=False)
    parser.add_argument('--out', dest='outFile', required=False)
    # parse args
    args = parser.parse_args()
    # set the output file
```

```
outFile = 'as.png'

if args.outFile:

    outFile = args.outFile

# set tile

tileFile = False

if args.tileFile:

    tileFile = Image.open(args.tileFile)
```

As with previous projects, you define the command line options for the program using `argparse`. The one required argument is the name of the depth map file ❶. There are also two optional arguments, one to provide an image file to use as the tile pattern and the other to set the name of the output file. If a tile image isn't specified, the program will generate a tile of random circles. If the output filename isn't specified, the autostereogram is written to a file named *as.png*.

Running the Autostereogram Generator

Now let's run the program using a depth map of a stool (*stool-depth.png*), which you'll find in the *data* folder of this project's GitHub repository:

```
$ python autos.py --depth data/stool-depth.png
```

Figure 8-8 shows the depth map image on the left and the generated autostereogram on the right. Because you haven't supplied a graphic for the tile, this autostereogram is created using random tiles.

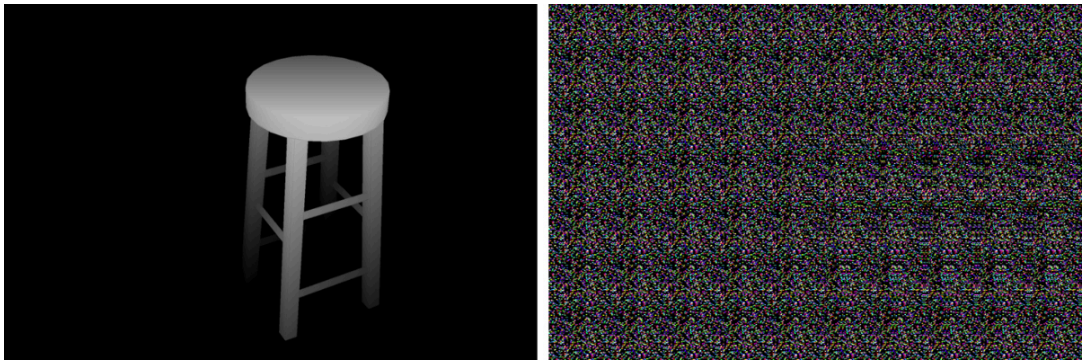


Figure 8-8: *A sample run of* autos.py

Now let's give a tile image as input. Use the *stool-depth.png* depth map as you did earlier, but this time, supply the image *escher-tile.jpg* for the tiles.

```
$ python autos.py --depth data/stool-depth.png --tile
data/escher-tile.jpg
```

Figure 8-9 shows the output.

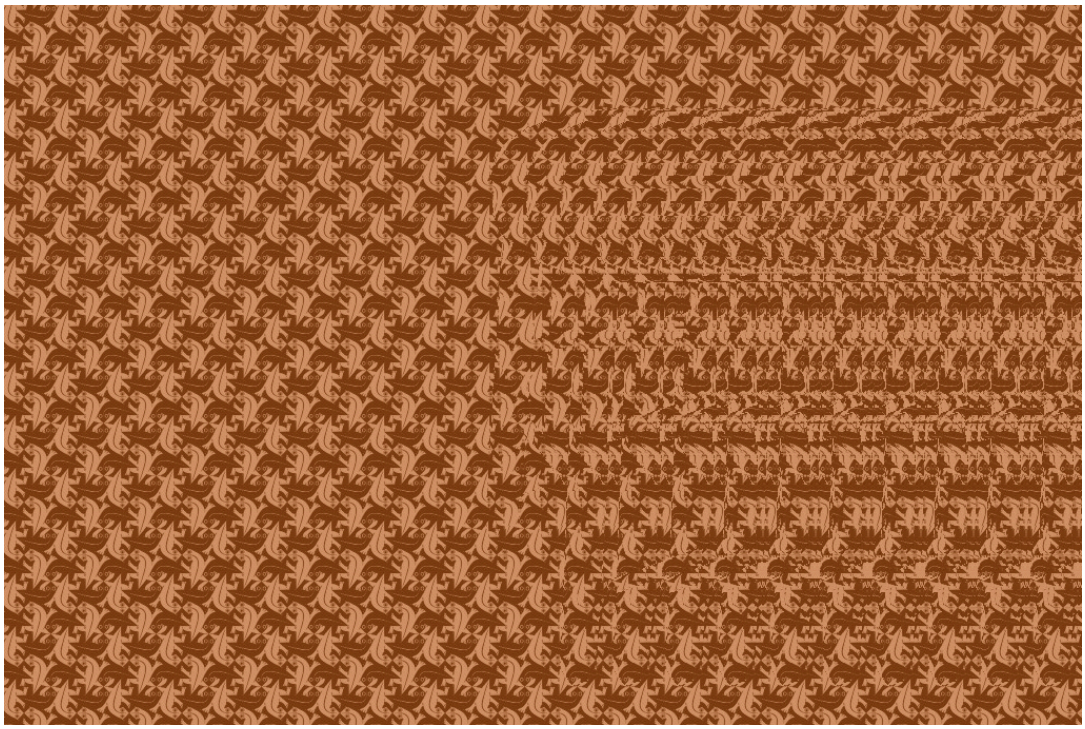


Figure 8-9: *A sample run of* autos.py *using tiles*

Enjoy creating moiré autostereograms using the images I've provided on GitHub or using your own depth maps!

Summary

In this project, you learned how to create autostereograms. Given a depth map image, you can now create either a random dot autostereogram or one tiled with an image you supply.

Experiments!

Here are some ways to further explore autostereograms:

1. Write code to create an image similar to **Figure 8-2** that demonstrates how changes in the linear spacing in an image can create illusions of depth. (Hint: use image tiles and the `Image.paste()` method.)
2. Add a command line option to the program to specify the scale to be applied to the depth map values. (Remember that the code divides the depth map value by 10.) How does changing the value affect the autostereogram?

The Complete Code

Here's the complete autostereogram project code:

"""

autos.py

A program to create autostereograms.

Author: Mahesh Venkitachalam

"""

```
import sys, random, argparse
from PIL import Image, ImageDraw
# create spacing/depth example
def createSpacingDepthExample():
    tiles = [Image.open('test/a.png'), Image.open('test/b.png'),
             Image.open('test/c.png')]
    img = Image.new('RGB', (600, 400), (0, 0, 0))
    spacing = [10, 20, 40]
    for j, tile in enumerate(tiles):
        for i in range(8):
            img.paste(tile, (10 + i*(100 + j*10), 10 + j*100))
    img.save('sdepth.png')
# create image filled with random dots
def createRandomTile(dims):
    # create image
    img = Image.new('RGB', dims)
    draw = ImageDraw.Draw(img)
    # calculate radius - % of min dimension
    r = int(min(*dims)/100)
```

```
    # number of dots
    n = 1000
    # draw random circles
    for i in range(n):
        # -r is used so circle stays inside - cleaner for tiling
        x, y = random.randint(r, dims[0]-r), random.randint(r, dims[1]-r)
        fill = (random.randint(0, 255), random.randint(0, 255),
                random.randint(0, 255))
        draw.ellipse((x-r, y-r, x+r, y+r), fill)
    # return image
    return img
# create a larger image of size dims by tiling the given image
def createTiledImage(tile, dims):
    # create output image
    img = Image.new('RGB', dims)
    W, H = dims
    w, h = tile.size
    # calculate # of tiles needed
    cols = int(W/w) + 1
```

```
    rows = int(H/h) + 1
    # paste tiles
    for i in range(rows):
        for j in range(cols):
            img.paste(tile, (j*w, i*h))
    # output image
    return img
# create a depth map for testing:
def createDepthMap(dims):
    dmap = Image.new('L', dims)
    dmap.paste(10, (200, 25, 300, 125))
    dmap.paste(30, (200, 150, 300, 250))
    dmap.paste(20, (200, 275, 300, 375))
    return dmap
# given a depth map (image) and an input image, create a new image
# with pixels shifted according to depth
def createDepthShiftedImage(dmap, img):
    # size check
    assert dmap.size == img.size
```

```
    # create shifted image
    sImg = img.copy()
    # get pixel access
    pixD = dmap.load()
    pixS = sImg.load()
    # shift pixels output based on depth map
    cols, rows = sImg.size
    for j in range(rows):
        for i in range(cols):
            xshift = pixD[i, j]/10
            xpos = i - 140 + xshift
            if xpos > 0 and xpos < cols:
                pixS[i, j] = pixS[xpos, j]
    # return shifted image
    return sImg
# given a depth map (image) and an input image, create a new image
# with pixels shifted according to depth
def createAutostereogram(dmap, tile):
    # convert depth map to single channel if needed
```

```
if dmap.mode != 'L':
    dmap = dmap.convert('L')
# if no tile specified, use random image
if not tile:
    tile = createRandomTile((100, 100))
# create an image by tiling
img = createTiledImage(tile, dmap.size)
# create shifted image
sImg = img.copy()
# get pixel access
pixD = dmap.load()
pixS = sImg.load()
# shift pixels output based on depth map
cols, rows = sImg.size
for j in range(rows):
    for i in range(cols):
        xshift = pixD[i, j]/10
        xpos = i - tile.size[0] + xshift
        if xpos > 0 and xpos < cols:
```

```
                pixS[i, j] = pixS[xpos, j]

    # return shifted image

    return sImg

# main() function

def main():

    # use sys.argv if needed

    print('creating autostereogram...')

    # create parser

    parser =
argparse.ArgumentParser(description="Autostereograms...")

    # add expected arguments

    parser.add_argument('--depth', dest='dmFile', required=True)

    parser.add_argument('--tile', dest='tileFile', required=False)

    parser.add_argument('--out', dest='outFile', required=False)

    # parse args

    args = parser.parse_args()

    # set output file

    outFile = 'as.png'

    if args.outFile:
```

```
        outFile = args.outFile

    # set tile

    tileFile = False

    if args.tileFile:

        tileFile = Image.open(args.tileFile)

    # open depth map

    dmImg = Image.open(args.dmFile)

    # create stereogram

    asImg = createAutostereogram(dmImg, tileFile)

    # write output

    asImg.save(outFile)

# call main

if __name__ == '__main__':

    main()
```

---

1 The hidden image is a shark.

---

2 ***http://colorstereo.com/texts_.txt/practice.htm***

3 *https://www.youtube.com/watch?v=oqpDqKpOChE*